Taller 1 — Aprendizaje Automático (2026-2S)

Datos, modelos lineales y decisiones bajo incertidumbre

Juan José Jiménez Maya

8 de octubre de 2026

Convenciones. Escribo $\tilde{X} = [\mathbf{1}_N \; X] \in \mathbb{R}^{N\times(D+1)}$ (la matriz X con una columna de unos al inicio), $\theta = (b, w^\top)^\top$ (intercepto y coeficientes en un solo vector) y $P = \mathrm{diag}(0, 1, \ldots, 1)$. Uso los mismos objetivos del notebook de clase [2]:

$$F_{\mathrm{lin}}(\theta) = \tfrac{1}{2N}\|y - \tilde{X}\theta\|_2^2 + \tfrac{\lambda_2}{2}\|w\|_2^2 + \lambda_1\|w\|_1, \qquad F_{\log}(\theta) = \tfrac{1}{N}\sum_n \big[\log(1+e^{z_n}) - y_n z_n\big] + \tfrac{\lambda_2}{2}\|w\|_2^2, \quad z_n = \tilde{x}_n^\top\theta.$$

Nunca penalizo el intercepto. El código está en el módulo `mlcore.py` y en tres notebooks ejecutados; las tablas y figuras que cito están en `resultados/`.

Alcance. Esta entrega incluye el Problema 1 de la parte teórica y los Problemas 4 a 7 de la parte práctica. **No presento los Problemas 2 y 3.** Me limito a las herramientas vistas en clase: mínimos cuadrados, Ridge, Lasso con umbral suave, regresión logística por descenso por gradiente, validación cruzada y el protocolo de evaluación del taller de vuelos.

Parte I. Fundamentación teórica

Problema 1. Mínimos cuadrados, Ridge y estabilidad

Pregunta central: ¿cómo puede aumentar el sesgo y, aun así, mejorar la estimación? La respuesta corta, que voy a justificar en (c), es que Ridge agrega un poco de sesgo pero quita mucha varianza en las direcciones donde los datos tienen poca información; si lo que se gana en varianza es más de lo que se pierde en sesgo, el error total baja.

(a) Verosimilitud, gradiente y ecuaciones normales.

Premisas. Supongo que cada respuesta es la predicción lineal más un ruido normal, y que las observaciones son independientes (es el modelo de las diapositivas 14–15 de la clase 5 [1]; la densidad normal está en [5, cap. 2]):

$$z = Xw + b\,\mathbf{1}_N, \qquad Y_n \mid x_n \sim \mathcal{N}(z_n, \sigma^2).$$

Paso 1: juntar b y w. Con $\tilde{X} = [\mathbf{1}_N \; X]$ y $\theta = (b, w^\top)^\top$ el producto da la predicción:

$$\tilde{X}\theta = \begin{bmatrix}\mathbf{1}_N & X\end{bmatrix}\begin{bmatrix}b\\ w\end{bmatrix} = \mathbf{1}_N b + Xw = z.$$

> **En palabras:** la columna de unos se multiplica por b, así que el intercepto queda "dentro" de la matriz y puedo tratar todo como un solo vector.

Paso 2: densidad de una observación. Es la normal evaluada en el valor observado y_n:

$$p(y_n) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\Big(-\frac{(y_n - z_n)^2}{2\sigma^2}\Big).$$

Paso 3: log-verosimilitud. Como las observaciones son independientes, las densidades se multiplican; al tomar logaritmo el producto pasa a ser una suma:

$$\ell(\theta) = \sum_{n=1}^{N}\Big[-\tfrac{1}{2}\log(2\pi\sigma^2) - \frac{(y_n - z_n)^2}{2\sigma^2}\Big] = -\frac{N}{2}\log(2\pi\sigma^2) - \frac{1}{2\sigma^2}\|y - \tilde{X}\theta\|_2^2.$$

Paso 4: relación con la pérdida. Defino $J(\theta) = \frac{1}{2N}\|y - \tilde{X}\theta\|_2^2$. Si multiplico ℓ por el número negativo $-\sigma^2/N$:

$$-\frac{\sigma^2}{N}\,\ell(\theta) = \underbrace{\tfrac{\sigma^2}{2}\log(2\pi\sigma^2)}_{\text{no depende de }\theta} + J(\theta).$$

Multiplicar por un negativo cambia "maximizar" por "minimizar" y sumar una constante no mueve el óptimo. Entonces **maximizar ℓ es lo mismo que minimizar J** [5, secc. 4.2]. Ridge agrega $\frac{\lambda_2}{2}\|w\|^2$, que equivale a la estimación MAP con una distribución previa $w \sim \mathcal{N}(0, \tau^2 I)$, con $\lambda_2 = \sigma^2/(N\tau^2)$ [5, secc. 4.5.3].

En palabras: si el ruido es normal, buscar los parámetros "más probables" es exactamente lo mismo que minimizar el error cuadrático medio.

Paso 5: expandir J. Uso $\|v\|^2 = v^\top v$:

$$J(\theta) = \tfrac{1}{2N}(y - \tilde{X}\theta)^\top (y - \tilde{X}\theta) = \tfrac{1}{2N}\big[y^\top y - y^\top \tilde{X}\theta - \theta^\top \tilde{X}^\top y + \theta^\top \tilde{X}^\top \tilde{X}\theta\big].$$

Paso 6: juntar los términos del medio. $y^\top \tilde{X}\theta$ es un número (1×1) y un número es igual a su transpuesta, así que $y^\top \tilde{X}\theta = \theta^\top \tilde{X}^\top y$:

$$J(\theta) = \tfrac{1}{2N}\big[y^\top y - 2\,\theta^\top \tilde{X}^\top y + \theta^\top \tilde{X}^\top \tilde{X}\theta\big].$$

Paso 7: derivar. Uso las identidades de la diapositiva 9 de la clase 5 [1]: $\nabla(\theta^\top a) = a$ y, si A es simétrica, $\nabla(\theta^\top A\theta) = 2A\theta$. Como $\tilde{X}^\top \tilde{X}$ es simétrica:

$$\nabla J(\theta) = \tfrac{1}{2N}\big[-2\tilde{X}^\top y + 2\tilde{X}^\top \tilde{X}\theta\big] = \tfrac{1}{N}\tilde{X}^\top(\tilde{X}\theta - y).$$

Paso 8: término de Ridge. El 0 de P "apaga" el intercepto: $\theta^\top P\theta = w_1^2 + \cdots + w_D^2 = \|w\|_2^2$. P también es simétrica, así que $\nabla\big(\frac{\lambda_2}{2}\theta^\top P\theta\big) = \lambda_2 P\theta$.

Paso 9: gradiente y Hessiano. Sumo los pasos 7 y 8 y derivo otra vez:

$$\nabla F(\theta) = \tfrac{1}{N}\tilde{X}^\top(\tilde{X}\theta - y) + \lambda_2 P\theta, \qquad \nabla^2 F(\theta) = \tfrac{1}{N}\tilde{X}^\top\tilde{X} + \lambda_2 P.$$

Paso 10: ecuaciones normales. En el mínimo el gradiente vale cero. Multiplico por N y saco θ como factor común:

$$\tilde{X}^\top(\tilde{X}\theta - y) + N\lambda_2 P\theta = 0 \quad \Longrightarrow \quad \boxed{(\tilde{X}^\top\tilde{X} + N\lambda_2 P)\,\theta = \tilde{X}^\top y}.$$

Con $\lambda_2 = 0$ quedan las ecuaciones de OLS; con $\tilde{X} = X$ y $P = I$ es la fórmula de Ridge de la diapositiva 17 [1]. *Dimensiones:* $\tilde{X}$ es $N \times (D+1)$; θ y ∇F son $(D+1) \times 1$; y y $\tilde{X}\theta$ son $N \times 1$; $\tilde{X}^\top\tilde{X}$, P y $\nabla^2 F$ son $(D+1) \times (D+1)$.

En palabras: Ridge es igual que OLS pero sumando $N\lambda_2$ en la diagonal (menos en la posición del intercepto). Esa suma es lo que hace que el sistema siempre tenga solución única, como muestro en (b).

(b) Unicidad y residuos.

Paso 1: suponer dos soluciones. Si θ_1 y θ_2 cumplen $\tilde{X}^\top\tilde{X}\theta = \tilde{X}^\top y$, al restarlas con $v = \theta_1 - \theta_2$ queda $\tilde{X}^\top\tilde{X}v = 0$.

Paso 2: multiplicar por $v^\top$. Así aparece una norma al cuadrado, que solo vale 0 si el vector es 0:

$$v^\top\tilde{X}^\top\tilde{X}v = (\tilde{X}v)^\top(\tilde{X}v) = \|\tilde{X}v\|_2^2 = 0 \quad \Longrightarrow \quad \tilde{X}v = 0.$$

Paso 3: conclusión para OLS. Si las columnas de $\tilde{X}$ son independientes (rango $D+1$, lo que exige $N \geq D+1$), $\tilde{X}v = 0$ solo pasa con $v = 0$ y la solución es única. Si alguna columna es combinación de otras, hay un $v \neq 0$ y $\theta + v$ también es solución. **OLS es única si y solo si $\tilde{X}$ tiene columnas independientes** (diapositiva 11 de [1]).

Paso 4: Ridge. Repito el paso 2 con la matriz de Ridge:

$$v^\top(\tilde{X}^\top\tilde{X} + N\lambda_2 P)v = \|\tilde{X}v\|_2^2 + N\lambda_2\|v_w\|_2^2 = 0.$$

Los dos sumandos son ≥ 0, así que los dos valen cero. Del segundo, $v_w = 0$; reemplazando en el primero, $\tilde{X}v = \mathbf{1}_N v_b = 0$, y entonces $v_b = 0$ porque $N \geq 1$. Luego $v = 0$: **Ridge con $\lambda_2 > 0$ tiene solución única aunque X tenga columnas dependientes y aunque b no se penalice.**

Paso 5: residuos. Con $r = y - \tilde{X}\hat{\theta}$, las ecuaciones normales dicen: en OLS, $\tilde{X}^\top r = 0$; en Ridge, $\tilde{X}^\top r = N\lambda_2 P\hat{\theta}$. En Ridge la primera fila (la de unos) sigue dando $\sum_n r_n = 0$ porque P tiene un 0 ahí, pero las demás dan $X^\top r = N\lambda_2\hat{w}$, que no es cero. Una proyección ortogonal necesita que el residuo sea perpendicular a *todas* las columnas. Por eso OLS sí es una proyección ortogonal (diapositivas 12–13 de [1]) y **no es correcto llamar proyección ortogonal al ajuste Ridge**: es un ajuste "encogido" (la matriz $H_\lambda = \tilde{X}(\tilde{X}^\top\tilde{X} + N\lambda_2 P)^{-1}\tilde{X}^\top$ no cumple $H_\lambda^2 = H_\lambda$).

En palabras: OLS deja un residuo que no tiene "nada más que explicar" con las columnas de X. Ridge, a propósito, deja algo sin explicar porque prefiere coeficientes más pequeños.

(c) Sesgo y varianza. Aquí "sesgo" es el error promedio del estimador, no el intercepto b. Uso la descomposición ECM = varianza + sesgo^2 de [5, secc. 4.7.6].

Premisas. $b = 0$, X fija y de rango completo, $y = Xw^\star + \varepsilon$ con $\mathbb{E}[\varepsilon] = 0$ y $\mathrm{Cov}(\varepsilon) = \sigma^2 I_N$.

Paso 1: solución. Con $b = 0$, P es la identidad y $\hat{w} = AX^\top y$ con $A = (X^\top X + N\lambda_2 I)^{-1}$ (simétrica).

Paso 2: valor esperado. La esperanza pasa por las matrices fijas y $\mathbb{E}[y] = Xw^\star$: $\mathbb{E}[\hat{w}] = AX^\top Xw^\star$.

Paso 3: sesgo. Despejo $X^\top X = A^{-1} - N\lambda_2 I$, entonces $AX^\top X = I - N\lambda_2 A$ y

$$\text{sesgo} = \mathbb{E}[\hat{w}] - w^\star = (I - N\lambda_2 A)w^\star - w^\star = -N\lambda_2 Aw^\star.$$

Paso 4: covarianza. Con $\mathrm{Cov}(My) = M\,\mathrm{Cov}(y)M^\top$ y $M = AX^\top$: $\mathrm{Cov}(\hat{w}) = AX^\top(\sigma^2 I)XA = \sigma^2 AX^\top XA$.

Paso 5: direcciones propias. Escribo $X^\top X = \sum_k d_k v_k v_k^\top$; $d_k > 0$ mide cuánta información hay en la dirección v_k. Con $\kappa = N\lambda_2$ y $\alpha_k = v_k^\top w^\star$, $A = \sum_k \frac{1}{d_k+\kappa} v_k v_k^\top$ y todo se separa por direcciones:

$$\text{sesgo}_k = -\frac{\kappa}{d_k + \kappa}\,\alpha_k, \qquad \mathrm{Var}_k = \sigma^2 \frac{d_k}{(d_k+\kappa)^2}.$$

En palabras: en cada dirección, Ridge multiplica la estimación de OLS por $d_k/(d_k+\kappa)$, un número entre 0 y 1. Si d_k es grande (mucha información) casi no la toca; si d_k es pequeño (variables casi colineales) la encoge mucho.

Paso 6: extremos. Si $\lambda_2 \to 0$: sesgo $\to 0$ y $\mathrm{Var}_k \to \sigma^2/d_k$ (OLS), que es enorme cuando d_k es pequeño. Si $\lambda_2 \to \infty$: $\hat{w} \to 0$, el sesgo tiende a $-\alpha_k$ y la varianza a 0.

Paso 7: ¿cuándo gana Ridge? Comparo el error de la dirección k (sesgo^2 + varianza) con el de OLS:

$$\frac{\kappa^2\alpha_k^2 + \sigma^2 d_k}{(d_k+\kappa)^2} < \frac{\sigma^2}{d_k}.$$

Multiplico en cruz, cancelo $\sigma^2 d_k^2$ a ambos lados y divido por $\kappa > 0$:

$$\kappa\,(d_k\alpha_k^2 - \sigma^2) < 2\sigma^2 d_k.$$

Dos lecturas: si κ es pequeño la desigualdad siempre se cumple, así que **siempre hay un λ_2 que mejora a OLS**. Pero si la dirección tiene señal fuerte ($d_k\alpha_k^2 > \sigma^2$) y κ es grande, la desigualdad falla: *una menor varianza no garantiza un menor error para cualquier regularización*, porque el sesgo^2 sigue creciendo. **Conclusión de estabilidad:** como $\mathrm{Var}_k \le \sigma^2/(4N\lambda_2)$, Ridge pone un techo a la inestabilidad de las direcciones casi colineales. Esto lo compruebo con la simulación del Problema 5.

Parte II. Implementación y experimentación

Problema 4. Modelos desde cero y verificación numérica

(a) Implementación.

Premisas. Todos los modelos están en un solo módulo, `mlcore.py`, hecho con NumPy: sin inversas explícitas, sin diferenciación automática y sin optimizadores genéricos. Cada clase separa entrenar (`fit`), predecir (`predict`, `predict_proba`) y evaluar (`objective`, `log_loss`).

Paso 1: fórmulas cerradas. OLS usa `np.linalg.lstsq`; si hay columnas repetidas da la solución de norma mínima y guardo el rango. Ridge resuelve con `np.linalg.solve` el sistema $(\tilde{X}^\top\tilde{X} + N\lambda_2 P)\theta = \tilde{X}^\top y$ del Problema 1(a), como el Algoritmo 1 de la clase 5 (resolver el sistema, no invertir la matriz) [1].

Paso 2: descenso por gradiente. Para regresión lineal y logística (con L2 opcional) actualizo $\theta \leftarrow \theta - \eta\nabla F(\theta)$, como `LinearRegressionScratch` y `LogisticRegressionScratch` del notebook de clase [2] y el Algoritmo 2 de [1]. En lugar de una tasa fija a mano uso $\eta = 1/L$, donde L es el mayor autovalor del Hessiano: $L = \sigma_{\text{máx}}(\tilde{X})^2/N + \lambda_2$ en el caso lineal y $L = \sigma_{\text{máx}}(\tilde{X})^2/(4N) + \lambda_2$ en el logístico, porque $p_n(1-p_n) \le 1/4$ en el Hessiano $\frac{1}{N}\tilde{X}^\top S\tilde{X}$ de la diapositiva 29. Con este paso el objetivo baja en cada iteración.

Paso 3: Lasso por gradiente proximal. Es la actualización de `fit_regularized` de clase [2]. El término $\lambda_1\|w\|_1$ no se puede derivar en cero, así que en cada iteración hago dos cosas: (1) un paso de gradiente sobre la parte derivable, $u = \theta - \eta\nabla f(\theta)$; (2) aplico a cada w_j el *umbral suave* $S(u_j, \eta\lambda_1) = \text{sign}(u_j)\,\text{máx}(|u_j| - \eta\lambda_1, 0)$, sin tocar b. Si $|u_j| \le \eta\lambda_1$ el coeficiente queda en *cero exacto*: los ceros salen del algoritmo y no de redondear al final.

Paso 4: parada y registro. Paro cuando $\|\theta^+ - \theta\|_2/\eta <$ `tol` (el criterio de `fit_regularized`). Sin L1 esto es exactamente la norma del gradiente; en Lasso es el residuo de la actualización proximal, que incluye a b. La tolerancia por defecto es 10^{-8} y el máximo va de $5 \cdot 10^4$ a $2 \cdot 10^5$ iteraciones. Guardo el objetivo completo en cada iteración, el número de iteraciones y el motivo de parada; reviso dimensiones, valores finitos e hiperparámetros; y cada `fit` vuelve a empezar en $\theta = 0$ (no uso arranque en caliente).

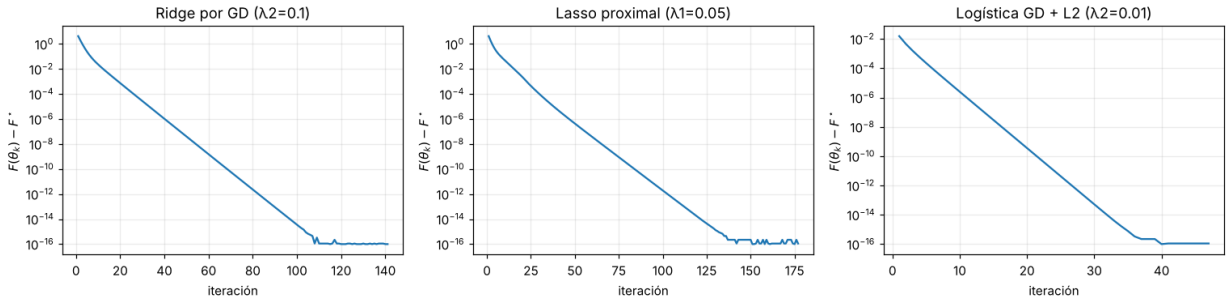


Figura 1: Historial de convergencia ($\eta = 1/L$, parada por tolerancia): Ridge por GD frente a su mínimo exacto, Lasso proximal y logística L2.

Conclusión. En la Figura 1 las tres corridas terminan por tolerancia y el objetivo baja en cada iteración. En Ridge la distancia al mínimo exacto (el de `solve`) baja hasta el error de redondeo, lo que confirma a la vez el gradiente y las ecuaciones normales.

(b) Pruebas automáticas.

Premisas. Programé 30 pruebas dentro del Notebook 1. Cada una compara un valor con un umbral fijo, y un `assert` detiene el notebook si alguna falla. La tabla completa está en `p4_pruebas_numericas.csv`; las 30 pasan.

Paso 1: gradientes. Comparo el gradiente que programé con el de diferencias centrales, $g_j \approx [F(\theta + he_j) - F(\theta - he_j)]/(2h)$ con $h = 10^{-6}$, en tres puntos $\theta \sim U(-1, 1)$, incluyendo b y L2. El peor caso es $\delta_g \leq 2.5 \times 10^{-10}$, muy por debajo de la meta 10^{-5}. La parte L1 no la derivo numéricamente en cero.

Paso 2: estabilidad. Evalúo la sigmoide y la pérdida en $z \in \{\pm 1000, \pm 100, \pm 1, 0\}$ con las dos etiquetas. Uso las mismas fórmulas del notebook de clase [2]: $\sigma(z) = \exp(-\log(1 + e^{-z}))$ y la pérdida $\log(1 + e^{-z})$ si $y = 1$ o $\log(1 + e^{z})$ si $y = 0$, calculadas con `np.logaddexp(0,u)`$= \log(e^0 + e^u)$, que nunca evalúa una exponencial gigante. Todos los valores salen finitos y el error frente al valor exacto es 0, sin recortar el score.

Paso 3: casos difíciles. OLS con una columna duplicada detecta rango D, cumple $\|\tilde{X}^\top r\| = 7 \times 10^{-14}$ y conserva las predicciones. Ridge con la misma columna duplicada tiene residuo de ecuaciones normales 4×10^{-14} y le da el mismo coeficiente a las dos copias, como dice 1(b). Lasso en un diseño ortogonal ($X^\top X/N = I$) reproduce el umbral suave $S(s, \lambda_1)$ con $s = X^\top y/N$ (error 9×10^{-16}); en un diseño correlacionado verifico la optimalidad con el residuo de la actualización proximal, $\|\theta^+ - \theta\|/\eta = 9 \times 10^{-12}$ (la solución es un punto fijo del algoritmo).

Paso 4: guardado y probabilidades. Guardar y cargar los parámetros en JSON no cambia las predicciones; las probabilidades están en $[0, 1]$ y suman 1; y cambiar τ no cambia `predict_proba` ni la log-loss.

(c) Comparación con scikit-learn 1.9.1.

Premisas. Comparo con scikit-learn [7] sobre las mismas matrices ya preparadas. Como scikit-learn escribe los objetivos de otra forma, multiplico mi objetivo por una constante para que queden iguales (Tabla 1); es la tabla de equivalencias del notebook de clase [2], con la derivación explícita.

Tabla 1: Correspondencia entre mis hiperparámetros y los de scikit-learn 1.9.1 (intercepto nunca penalizado).

Estimador	Objetivo de scikit-learn	Multiplico el mío por	Equivalencia
`LinearRegression`	$\|y - Xw - b\|^2$	$2N$	sin penalización
`Ridge`	$\|y - Xw - b\|^2 + \alpha\|w\|^2$	$2N$	$\alpha = N\lambda_2$
`Lasso`	$\frac{1}{2N}\|y - Xw - b\|^2 + \alpha\|w\|_1$	1	$\alpha = \lambda_1$
`LogisticRegression`	$C\sum_n \ell_n + \frac{1}{2}\|w\|^2$	$1/(N\lambda_2)$	$C = 1/(N\lambda_2)$

Ejemplo de la derivación (Ridge). Mi objetivo es $\frac{1}{2N}\|r\|^2 + \frac{\lambda_2}{2}\|w\|^2$. Al multiplicarlo por $2N$ queda $\|r\|^2 + N\lambda_2\|w\|^2$, que es el de scikit-learn con $\alpha = N\lambda_2$. Las demás filas salen igual.

Paso 1: recalcular con N y $N/2$. Como α de Ridge y C dependen de N, repito todo con N y con $N/2$ filas, recalculando la equivalencia. Para la logística sin penalizar uso `C=np.inf`.

Paso 2: resultados. En los 12 contrastes la diferencia relativa de objetivo es como mucho 2.2×10^{-16} y el error relativo de predicciones o probabilidades como mucho 9.7×10^{-9}, frente a la meta 10^{-4}. En la logística comparo probabilidades, no solo etiquetas.

Paso 3: caso sin solución única. Con una columna duplicada, Lasso tiene infinitas soluciones (solo la suma de los dos coeficientes queda determinada). scikit-learn da $(0.58, 0.89)$ y mi gradiente proximal da $(0.73, 0.73)$: coeficientes distintos, pero el mismo objetivo y las mismas predicciones (error 3.6×10^{-12}).

Conclusión. Mis implementaciones coinciden con scikit-learn a precisión de máquina. Cuando la solución no es única, comparar coeficientes uno a uno daría una falsa alarma; lo correcto es comparar el objetivo y las

predicciones.

Problema 5. Simulación de sesgo, varianza y selección

En palabras: simulo muchas veces el mismo experimento cambiando solo el ruido. Así puedo medir cuánto se equivoca un modelo "en promedio" (sesgo) y cuánto cambia de una réplica a otra (varianza), cosa que con un solo conjunto de datos no se puede ver.

(a) Diseño.

Premisas. Sigo el enunciado: $N = 120$, $D = 20$, $M = 1000$, $R = 40$, $\sigma^2 = 1$, filas $x_n \sim \mathcal{N}(0, \Sigma_\rho)$ [5, secc. 3.2] con cuatro bloques de 5 variables y correlación $\rho \in \{0, 0.9\}$ dentro de cada bloque, coeficientes verdaderos $w_1^\star = 3$, $w_6^\star = -2$, $w_{11}^\star = 1.5$, $w_{16}^\star = -1$ (uno por bloque), sin intercepto y sin escalador.

Paso 1: semillas. De la semilla maestra 2026 saco subsemillas para X, para X_* y para cada uno de los 40 ruidos, y las guardo en `p5_subsemillas.json`. Así cada réplica se puede repetir por separado.

Paso 2: ajustes. Con X y X_* fijas, ajusto OLS, Ridge y Lasso con mis clases para cada $\lambda \in \{10^{-3}, \dots, 10\}$ y cada una de las 40 respuestas.

(b) Descomposición del error.

Definiciones. En cada punto de auditoría m, $f_m^\star$ es el valor verdadero, $\hat{f}_{rm}$ la predicción de la réplica r y $\bar{f}_m$ el promedio de las 40. $\hat{B}^2$ mide qué tan lejos está el promedio de la verdad, $\hat{V}$ cuánto se dispersan las réplicas alrededor de su promedio y $\hat{E}_f$ el error total.

Paso 1: identidad exacta. Para cada m, al expandir $(\hat{f}_{rm} - f_m^\star)^2 = \big((\hat{f}_{rm} - \bar{f}_m) + (\bar{f}_m - f_m^\star)\big)^2$ aparece el término cruzado $\frac{2}{R}\sum_r(\hat{f}_{rm} - \bar{f}_m)(\bar{f}_m - f_m^\star)$, que es exactamente cero porque $\sum_r(\hat{f}_{rm} - \bar{f}_m) = 0$. Esto solo es exacto con el factor $1/R$ (con $1/(R-1)$ no cierra). Es la misma cuenta de ECM = varianza + sesgo^2 de [5, secc. 4.7.6], hecha con promedios sobre réplicas. Numéricamente $\hat{E}_f = \hat{B}^2 + \hat{V}$ hasta el error de redondeo.

Paso 2: respuesta nueva. Si quiero predecir una respuesta nueva $y_* = f^\star + \varepsilon_*$, el ruido ε_* es independiente de todo lo anterior y nadie lo puede predecir, así que el error esperado es $\hat{E}_f + \sigma^2$: por eso sumo $\sigma^2 = 1$.

Paso 3: efecto de tener solo 40 réplicas. $\bar{f}_m$ es un promedio de 40 réplicas, así que también tiene algo de ruido. Por eso OLS, que es insesgado, muestra $\hat{B}^2 \approx 0.004$–0.007 en vez de 0.

Tabla 2: Error en X_* (en negrita, el menor $\hat{E}_f$ de cada modelo y escenario). OLS: $\hat{E}_f = 0.201$ ($\rho = 0$) y 0.200 ($\rho = 0.9$), con $\hat{V} \approx 0.195$.

	$\rho = 0$						$\rho = 0.9$					
	Ridge			Lasso			Ridge			Lasso		
λ	$\hat{B}^2$	$\hat{V}$	$\hat{E}_f$	$\hat{B}^2$	$\hat{V}$	$\hat{E}_f$	$\hat{B}^2$	$\hat{V}$	$\hat{E}_f$	$\hat{B}^2$	$\hat{V}$	$\hat{E}_f$
10^{-3}	.007	.194	.201	.007	.191	.198	.003	.191	.195	.004	.186	.190
10^{-2}	.008	.189	**.197**	.006	.164	.170	.016	.160	**.176**	.005	.123	.128
10^{-1}	.194	.150	.344	.043	.053	**.096**	.384	.065	.449	.050	.054	**.104**
1	4.36	.039	4.40	4.65	.032	4.68	1.45	.021	1.48	2.76	.032	2.79
10	13.2	.001	13.2	16.0	.000	16.0	7.26	.003	7.27	15.8	.000	15.8

Paso 4: lectura de la tabla. Al subir λ, $\hat{V}$ baja siempre y $\hat{B}^2$ sube. El mejor $\hat{E}_f$ está en un λ intermedio: Ridge con $\lambda_2 = 0.01$ y Lasso con $\lambda_1 = 0.1$, ambos mejores que OLS (0.20). Con $\lambda \geq 1$ el sesgo domina y los dos son mucho peores que OLS.

Paso 5: Ridge frente a la teoría. Comparo con las fórmulas de 1(c). La media empírica se separa de $AX^\top X w^\star$ lo mismo que se espera por tener solo 40 réplicas (del orden de $\sqrt{\operatorname{tr}\operatorname{Cov}/R}$), y la varianza total empírica tr Cov sigue a la teórica $\operatorname{tr}(\sigma^2 AX^\top XA)$ (por ejemplo, 1.59 frente a 1.50 con $\rho = 0.9$, $\lambda_2 = 10^{-3}$) y baja igual que ella al subir λ_2. Con $\rho = 0.9$ el número de condición de $X^\top X$ es 145 (4.5 con $\rho = 0$). En la dirección de menor autovalor ($d = 5.1$) OLS tiene varianza 0.194, y $\lambda_2 = 0.01$ la baja a 0.128 a cambio de un sesgo de 0.33; en la dirección más grande ($d = 747$) el sesgo casi no aparece. Es justo lo que predice 1(c): Ridge encoge sobre todo las direcciones inestables.

(c) Selección con Lasso.

Premisas. Fijo de antemano la tolerancia $|\hat{w}_j| > 10^{-8}$ para decir que una variable fue elegida, y no la cambio después. Precisión = fracción de elegidas que son relevantes; sensibilidad = fracción de las 4 relevantes que fueron elegidas. Un bloque se "recupera" si elijo al menos una de sus cinco variables.

Paso 1: sin correlación. Con $\rho = 0$, $\lambda_1 = 1$ acierta el soporte exacto en el 82.5 % de las réplicas (precisión 1, sensibilidad 0.96), pero su $\hat{E}_f$ es 4.68. En cambio $\lambda_1 = 0.1$, el mejor para predecir, elige 8.4 variables en promedio (precisión 0.49, sensibilidad 1) y nunca acierta el soporte exacto.

Paso 2: con correlación. Con $\rho = 0.9$ y $\lambda_1 = 1$, la sensibilidad individual baja a 0.76 mientras que la de bloque es 1.0, con 0.98 sustituciones por réplica: Lasso elige otra variable del mismo bloque en lugar de la verdadera. Ningún λ acierta el soporte exacto, y $\lambda_1 = 10$ pone todos los coeficientes en cero.

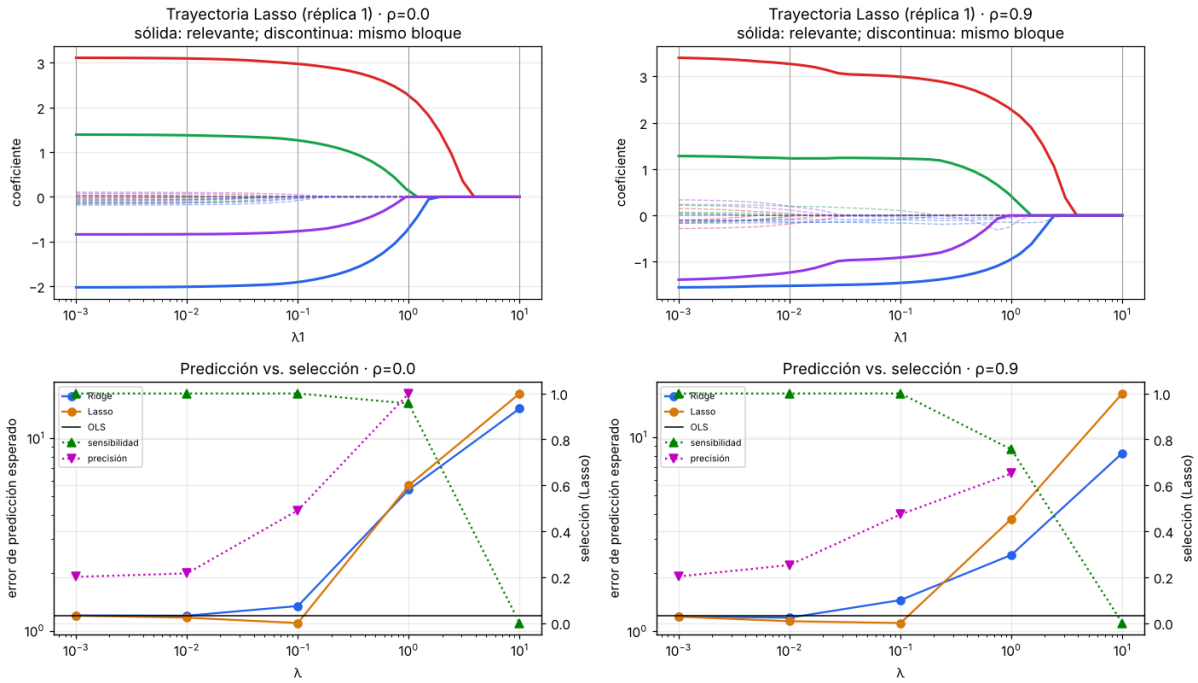


Figura 2: Arriba: trayectorias de Lasso (réplica 1); línea sólida = variable relevante, discontinua = otra variable de su bloque. Abajo: error de predicción esperado frente a sensibilidad y precisión.

(d) Respuestas con evidencia.

¿La regularización reduce sesgo y varianza a la vez? No. En la Tabla 2, al subir λ la varianza baja y el sesgo^2 sube. La única bajada conjunta (Lasso, $\rho = 0$, de 10^{-3} a 10^{-2}: $\hat{B}^2$ de 0.007 a 0.006) es del tamaño del ruido de promediar 40 réplicas. La regularización *cambia* varianza por sesgo, y solo mejora el error total en un rango intermedio.

¿Mejor predicción implica recuperar el soporte? Tampoco. El mejor Lasso para predecir nunca acierta el soporte exacto, y el que mejor acierta el soporte predice unas 49 veces peor. Elegir variables y predecir son objetivos distintos.

¿Cuándo una irrelevante reemplaza a una relevante? Cuando está muy correlacionada con ella ($\rho = 0.9$). En la muestra $x_j \approx x_1$, así que $w_j x_j$ aporta casi lo mismo con una penalización parecida. La variable es irrelevante en el modelo verdadero pero útil para predecir, y por eso la selección cambia entre réplicas.

Problema 6. Regresión reproducible sobre Airfoil

(a) Auditoría, partición y EDA.

Premisas. Cargo los datos [6] con la misma descarga y los mismos nombres de columnas del notebook de clase [2]. Conservo las 1503 filas crudas y uso como `id` el número de línea (`AF0001`, ...). Antes de partir solo aplico reglas fijas de formato y dominio, sin calcular estadísticas.

Paso 1: auditoría. Todas las columnas son numéricas; no hay faltantes, valores fuera de dominio (todo positivo y ángulo ≥ 0), duplicados exactos ni filas con predictores repetidos. No elimino ninguna fila (si hubiera duplicados exactos los quitaría antes de partir, como en el notebook de clase, para que no queden repartidos).

Paso 2: partición. Hago la partición 80/20 con `train_test_split` y semilla 2026. Quedan 1202 filas de desarrollo y 301 de prueba (`p6_particiones.csv`, por `id`).

Paso 3: EDA solo en desarrollo. Sigo el flujo del notebook de EDA de clase [3] (pregunta e hipótesis antes de mirar, univariado, asociación, grupos con tamaño y regla IQR solo para diagnóstico). Planteé tres hipótesis:

- *H1: el sonido baja con la frecuencia y es más lineal en escala log.* Se rechaza en parte: la media por frecuencia forma una U invertida (124.4 dB a 200 Hz, 128.7 dB a 1 kHz y 118.3 dB a 10 kHz), y la correlación con $\log_{10} f$ (-0.361) es incluso más débil que con f (-0.390).
- *H2: espesor y ángulo son redundantes.* Están muy asociados (Spearman 0.885). El logaritmo del espesor casi no cambia su relación con el sonido, así que no lo transformo.

- *H3: más velocidad, más sonido.* Por velocidad ($n = 217/376/232/377$) las medias van de 123.5 a 125.9 dB: el efecto es pequeño.

Conclusión. Los valores fuera de los bigotes (67 en frecuencia, 102 en espesor) son parte del diseño del experimento, así que los dejo. Ninguna decisión usa estadísticas de prueba.

(b) Validación cruzada.

Premisas. Dos representaciones. La A usa las cinco variables estandarizadas. La B, motivada por H1, cambia f por $\log_{10}(f/1\,\text{Hz})$ y $(\log_{10}(f/1\,\text{Hz})-3)^2$; dominio $f > 0$, unidades $\log_{10}$Hz, y el centro 3 (1000 Hz) es una constante que fijé, no se estima. El cuadrado permite la U invertida y el modelo sigue siendo lineal en sus parámetros.

Paso 1: protocolo. Validación cruzada de $K = 5$ pliegues [5, secc. 4.5.5]: `KFold(5, shuffle, 2026)`, los mismos pliegues para todos los candidatos y el escalador ajustado dentro de cada pliegue (si lo ajustara con todo el desarrollo, el pliegue de validación "vería" sus propias medias: fuga). Antes de comparar fijé la grilla $\{10^{-3}, \ldots, 10\}$ y la regla de desempate: si dos candidatos difieren en menos de 0.001 dB prefiero la representación A, luego el mayor λ y luego Ridge.

Tabla 3: RMSE (dB) en CV de 5 pliegues: media (desviación entre pliegues). Tabla completa en `p6_cv_resumen.csv`.

Rep.	Media	OLS	Ridge 10^{-2}	Ridge 10^{-1}	Lasso 10^{-2}	Lasso 1
A	6.889 (0.28)	4.778 (0.21)	4.778 (0.21)	4.827 (0.18)	4.778 (0.21)	5.520 (0.20)
B	—	4.611 (0.19)	**4.611 (0.19)**	4.650 (0.18)	4.612 (0.19)	5.413 (0.22)

Paso 2: lectura de la tabla. La representación B mejora entre 0.07 y 0.25 dB en los cinco pliegues. OLS, Ridge y Lasso con $\lambda \leq 0.01$ empatan a menos de 0.001 dB: con unas 960 filas y 6 variables estandarizadas el problema está bien condicionado (justo lo contrario del caso $\rho = 0.9$ del Problema 5), así que no hay varianza que corregir.

Conclusión. Con la regla fijada elijo **Ridge con $\lambda_2 = 0.01$ en la representación B**; el finalista OLS también usa B. La desviación entre pliegues no es un intervalo de confianza.

(c) Evaluación final única.

Paso 1: reajuste y prueba. Congelo representación e hiperparámetros, reajusto los finalistas con todo el desarrollo y evalúo una sola vez en prueba. Referencia: RMSE 6.93, MAE 5.65 (con la mediana), $R^2 = -0.003$. OLS-B: RMSE 4.924, MAE 3.694, $R^2 = 0.494$. Ridge-B: RMSE 4.921, MAE 3.700, $R^2 = 0.495$.

Paso 2: comparación con la CV. El RMSE de prueba es mayor que la media de CV (4.61) y que el peor pliegue (4.84). Una partición de 301 filas tiene su propia variabilidad; no reajusto nada por eso.

Paso 3: errores sistemáticos. La nube predicho vs. observado tiene pendiente menor que 1: sobrestimo los sonidos bajos y subestimo los altos, y los residuos forman una U invertida (Figura 3). Además el error es mucho mayor para $f \leq 1600$ Hz (RMSE 5.8) que para $f > 1600$ Hz (3.8), con segmentos definidos por la mediana de desarrollo. Las dos cosas indican que un modelo lineal con solo dos transformaciones no captura toda la relación (por ejemplo, posibles interacciones entre la frecuencia y las demás variables).

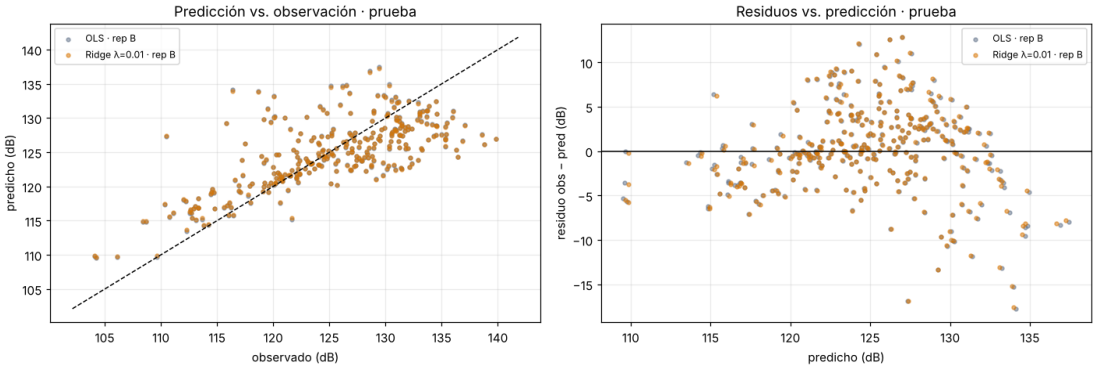


Figura 3: Airfoil, prueba: predicción frente a observación y residuos frente a predicción (OLS-B y Ridge-B casi se superponen).

(d) Escenario de generalización.

Paso 1: qué mide mi partición. Una partición aleatoria de filas mide *interpolación dentro del mismo experimento*: casi toda fila de prueba tiene otras frecuencias del mismo ensayo (misma cuerda, ángulo y velocidad) en desarrollo.

Paso 2: cómo evaluaría condiciones nuevas. Agruparía por ensayo (cuerda, ángulo y velocidad) con `GroupKFold`, o guardaría valores completos para prueba, como una velocidad o una cuerda nunca vistas. No hice esa evaluación, así que no afirmo esa capacidad.

Conclusión. La evidencia (empates en CV y en prueba, y Lasso sin anular coeficientes) dice que regularizar no ayuda aquí: elegí Ridge por la regla de desempate y OLS sería igual de defendible. La mejora real viene de la representación. Los coeficientes son asociaciones dentro de este diseño, no efectos causales.

Problema 7. Clasificación temporal y política de alertas

En palabras: quiero avisar, dos horas antes, qué vuelos se van a retrasar. Hay pocos retrasos (2–4 %), así que hay que cuidar tres cosas: no usar información del futuro, medir con métricas que sirvan para clases desbalanceadas, y elegir el umbral de alerta pensando en costos y en cuántas alertas se pueden atender.

(a) Datos, diccionario y partición.

Premisas. Uso `generar_vuelos(n=9600, seed=2026)` sin cambios y $Y = \mathbb{1}\{\texttt{min_retraso} \geq 15\}$. Todo el protocolo (partición temporal, *pipeline*, fuga, métricas, umbrales y evaluación final) sigue el taller de vuelos de clase [4].

Paso 1: diccionario. Dos horas antes se conocen la hora programada, la distancia, la lluvia y el viento *pronosticados*, la ocupación programada, el tráfico programado, la aerolínea y el origen. `min_retraso` se conoce después, así que solo es la etiqueta; `fecha_prediccion` solo sirve para ordenar y partir.

Paso 2: partición temporal. Ordeno por fecha, corto al 60 % y al 80 % y quito las 24 filas anteriores a cada corte. Entrenamiento V00000–V05735 (5736 filas, del 2024-01-01 al 2024-08-26), validación V05760–V07655 (1896) y prueba V07680–V09599 (1920). Con `assert` compruebo que hay 25 h entre conjuntos seguidos.

Paso 3: auditoría en entrenamiento. 140 retrasos (2.44 %) y cerca de 6 % de faltantes en lluvia, viento, ocupación y aerolínea; el tráfico y la prevalencia crecen con el tiempo. El preprocesamiento (mediana y estandarización; moda y one-hot con `handle_unknown='ignore'` para categorías nuevas [5, secc. 1.5.3 y 1.5.5]) se ajusta dentro de cada entrenamiento.

(b) Comparación por validación cruzada temporal.

Premisas. Comparo mi logística L2 sin ponderar (`LogisticRegressionGD`) con la ponderada de scikit-learn, `LogisticRegression(class_weight='balanced')`, como en el notebook de clase. Sus pesos son $a_c = N/(2N_c)$, calculados en cada entrenamiento, y suman $\sum_n a_{y_n} = N$. scikit-learn minimiza $C\sum_n a_{y_n}\ell_n + \frac{1}{2}\|w\|^2$ sin penalizar el intercepto; al dividir por CN queda $\frac{1}{N}\sum_n a_{y_n}\ell_n + \frac{\lambda_2}{2}\|w\|^2$ con $C = 1/(N\lambda_2)$, que es la misma traducción de la Tabla 1.

Paso 1: pliegues. `TimeSeriesSplit(4, gap=24)`: el pliegue de ajuste crece y siempre va antes del de evaluación. Los cuatro pliegues evaluados tienen las dos clases (27 a 31 positivos). Elijo por AP (área bajo precisión–recall), que se fija en los positivos y sirve con clases desbalanceadas.

Tabla 4: AP media (desviación) en CV temporal. Referencia constante: AP 0.026. No alertar: costo 0.31 por vuelo.

λ_2	10^{-3}	10^{-2}	10^{-1}	1	10
Sin ponderar	0.229 (0.06)	0.236 (0.08)	0.216 (0.08)	0.209 (0.08)	0.209 (0.08)
Ponderada	0.226 (0.07)	0.238 (0.08)	**0.246 (0.10)**	0.223 (0.09)	0.211 (0.08)

Conclusión. Gana la **logística ponderada con** $\lambda_2 = 0.1$, sin empate. Su ventaja (0.011) es pequeña frente a la dispersión entre pliegues: es una elección por regla, no una superioridad demostrada. Además la ponderación empeora la log-loss (0.46 frente a 0.10) y el Brier.

(c) Contraexperimento de fuga y diagnóstico en validación.

Paso 1: contraexperimento. Con el ganador y los pliegues fijos, ajusto el preprocesamiento una sola vez con todo el entrenamiento antes de la CV (contaminado) y lo comparo con ajustarlo en cada pliegue (correcto). El contaminado usa, por ejemplo, una mediana de tráfico de 48.21 en lugar de 46.18–47.43. Aun así, ΔAP (contaminada menos correcta) vale -0.002 en promedio (de -0.005 a $+0.0002$) y Δlog-loss -0.0008. Ni el signo ni el tamaño prueban que no haya fuga: la fuga es un error del *procedimiento* (usar información que no habría al predecir); aquí lo filtrado predice poco y la deriva temporal puede ayudar o empeorar. Estos resultados no participan en la selección.

Paso 2: métricas en validación. Prevalencia 3.9 %. Sin ponderar: AP 0.392, ROC-AUC 0.851, log-loss 0.126, Brier 0.032. Ponderado: AP 0.394, ROC-AUC 0.853, log-loss 0.535, Brier 0.178. La AP (suma por escalones) y el área trapezoidal bajo PR (une los puntos con rectas) difieren en cerca de 0.002.

Paso 3: calibración. Con 5 grupos por cuantiles (379–380 casos; 0/2/9/10/53 y 0/2/9/9/54 positivos; sin probabilidades repetidas), el modelo sin ponderar está cerca de la diagonal salvo en el último grupo (0.090 frente a 0.140). El ponderado sobrestima mucho (0.15–0.70 frente a 0–0.14): al darle a cada retraso un peso unas 40 veces mayor ($a_1/a_0 = N_0/N_1$), el modelo aprende probabilidades "infladas". Ordena bien, pero sus números no son probabilidades reales. No cambio el ganador.

(d) Política de alertas.

Umbral teórico. Si p fuera la probabilidad verdadera de retraso, alertar cuesta en promedio $1 \cdot (1-p)$ (falsa alarma) y no alertar cuesta $12p$ (retraso no avisado). Conviene alertar cuando $12p \geq 1-p$, es decir, $p \geq 1/13 \approx 0.077$. Esta regla va vuelo por vuelo y no limita cuántas alertas salen. Es la idea de la diapositiva 32 de [1]: cambiar τ cambia la decisión, no el modelo.

Paso 1: tabla de umbrales. Con las probabilidades de validación del ganador evalúo 1898 umbrales: todos los valores únicos, "alertar a todos" ($\tau = 0$) y "no alertar" ($\tau = +\infty$), con un indicador de modo extremo.

Paso 2: factibilidad. Ninguno cumple a la vez recall ≥ 0.80 y alertas $\leq 15\,\%$: el resultado es inviabilidad empírica. Con máximo 15 % de alertas el mejor recall es 0.635 ($\tau = 0.6351$, 14.0 %); para llegar a recall 0.80 hay que alertar al menos al 34.6 % ($\tau = 0.4546$).

Paso 3: referencia diagnóstica. Congelo como *referencia diagnóstica, no como política aprobada*, el umbral de menor costo sin restricciones, $\tau = 0.6351112239347184$ (costo 0.286).

Paso 4: comparación. $\tau = 0.5$ da recall 0.77 con 28 % de alertas (costo 0.358). $\tau = 1/13$ supone probabilidades correctas; con las probabilidades infladas del modelo ponderado alerta al 98.5 % (costo 0.946), peor que no alertar (0.468): lo que falla es el supuesto de probabilidades correctas, no la regla.

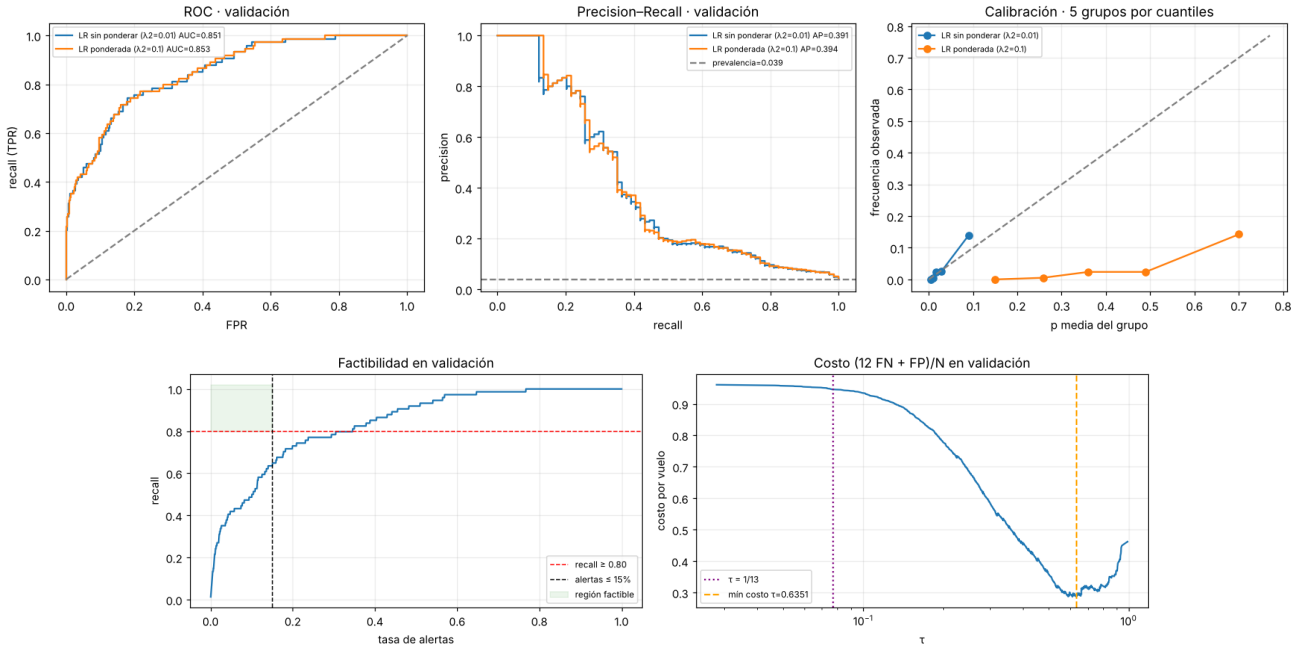


Figura 4: Vuelos, validación: ROC, PR y calibración (5 grupos por cuantiles) de los dos candidatos; factibilidad recall–alertas y costo por umbral del ganador.

(e) Evaluación final en prueba.

Premisas. Antes de abrir la prueba guardé un manifiesto JSON con variables, preprocesamiento, parámetros, regla y umbral sin redondear, y no reentreno con validación. La prueba tiene 71 retrasos en 1920 vuelos.

Paso 1: métricas de probabilidad. AP 0.409, ROC-AUC 0.869, log-loss 0.561 y Brier 0.191, iguales para las tres políticas porque no dependen del umbral (lo compruebo con `assert`).

Paso 2: métricas de decisión. Calculo TP, TN, FP y FN con NumPy y los verifico con `confusion_matrix` (Tabla 5).

Tabla 5: Evaluación única en prueba (alertas por cada 1000 vuelos).

Política	TP	FP	FN	TN	Prec.	Recall	Espec.	F1	Acc.	Costo / Alertas
$\tau = 0.5$	57	565	14	1284	0.092	0.803	0.694	0.165	0.698	0.382 / 324
$\tau = 1/13$	71	1821	0	28	0.038	1.000	0.015	0.072	0.052	0.948 / 985
$\tau = 0.6351$ (diagn.)	50	234	21	1615	0.176	0.704	0.873	0.282	0.867	0.253 / 148

Conclusión. Ninguna política cumple las dos restricciones en prueba. La diagnóstica respeta la capacidad (148 alertas por cada 1000) y tiene el menor costo, pero su recall es 0.70. Cumplir en validación no garantizaba cumplir en prueba: elegí el umbral con solo 74 positivos (cada uno mueve el recall cerca de 1.4 puntos) y la prevalencia y el tráfico cambian con el tiempo. No ajusto la política después de ver la prueba; concluyo que con estas variables la meta conjunta no se alcanza y que habría que revisar la capacidad o conseguir más información.

Conclusiones

- Mis implementaciones coinciden con scikit-learn a precisión de máquina cuando igualo los objetivos. Cuando la solución no es única, hay que verificar con el objetivo y las predicciones, no con los coeficientes.
- La regularización cambia varianza por sesgo: ayuda cuando los datos están mal condicionados (simulación con $\rho = 0.9$) y no aporta cuando el problema ya está bien condicionado (Airfoil), donde importa más la representación.
- Elegir las variables correctas y predecir bien son objetivos distintos.
- En clasificación desbalanceada, ordenar bien (AP, ROC-AUC) no significa tener probabilidades calibradas; el umbral de Bayes solo vale con probabilidades correctas, y si una restricción operativa no se puede cumplir, hay que reportarlo.

Reproducibilidad. En la última ejecución (Linux, Python 3.13), desde un kernel limpio, los notebooks 01, 02 y 03 tardan 24 s, 12 s y 24 s (medido con `jupyter nbconvert --execute`, incluyendo el arranque del kernel). Versiones y orden de ejecución en el README.

Atribución, contribuciones y uso de IA generativa

Código adaptado de `Regresion_lineal_logistica_regularizacion.ipynb` (notebook de clase).

- `LinearRegressionScratch` → `LinearRegressionGD` y su clase base `_Linear`. Conservé el descenso por gradiente sobre MSE/2 y la parada por la norma del gradiente. Cambié: el vector aumentado $\theta = (b, w)$ con la matriz P para agregar L2 sin penalizar b; la tasa `learning_rate` por $\eta = 1/L$ calculado con la norma espectral (por eso **eliminé el parámetro de tasa manual**, que ya no se usaba); el inicio en $\theta = 0$ en vez de $b = \bar{y}$; y agregué historial del objetivo, motivo de parada, revisión de entradas y guardado en JSON (Problema 4a–4b).
- `LogisticRegressionScratch` → `LogisticRegressionGD`. Mantuve la estructura scores–probabilidades–error–gradiente y agregué L2, `predict(X, threshold)` y `log_loss` calculada desde el score. Para el modelo ponderado del Problema 7 uso `LogisticRegression(class_weight='balanced')` de scikit-learn, como `construir_pipeline` del taller de vuelos.
- `fit_regularized` → función `gradiente_proximal` y clase `LassoRegression`. Tomé el paso de gradiente seguido del umbral suave y la parada por $\|\theta^+ - \theta\|/\eta$. Lo separé en una función que usan los tres modelos iterativos (para no repetir el algoritmo) y apliqué el umbral solo a w con $\eta\lambda_1$; agregué `proximal_residual` para verificar la optimalidad de Lasso.
- `sigmoid` y `binary_cross_entropy` → `sigmoid` (sin cambios) y `logistic_loss_terms`, que devuelve la pérdida de cada observación con la misma fórmula `np.logaddexp` en lugar de su promedio.
- `prepare_data` y la celda de descarga de Airfoil → `ajustar_predecir` y la celda de carga del Notebook 2. Reutilicé la URL, los nombres de columnas y la idea de ajustar el `StandardScaler` solo con entrenamiento; agregué el `id` estable, el manifiesto de la partición y los pliegues de validación cruzada (Problema 6).

Código adaptado de `Taller_clasificacion_vuelos.ipynb` (notebook de clase).

- `generar_vuelos`: copia literal, sin cambiar el mecanismo; solo paso `seed=2026` explícito.
- `separar_por_tiempo` → celda de partición del Notebook 3: mismos cortes 60/80 % y gap de 24 filas; agregué `id_vuelo`, el manifiesto `p7_particiones.csv` y la comprobación de las 25 h.
- `construir_pipeline` → `crear_prep`, `crear_modelo`, `ajustar` y `probas`: el mismo `ColumnTransformer`, separado del clasificador porque el modelo sin ponderar es mi `LogisticRegressionGD`; el ponderado es el mismo `LogisticRegression(class_weight='balanced')` de clase con $C = 1/(N\lambda_2)$.
- `comparar_fuga` → celda del contraexperimento: misma comparación global vs. por pliegue, con mi modelo, la log-loss y las estadísticas de tráfico usadas.
- `tabla_umbrales` y `seleccionar_umbral` → `tabla_umbrales` y la celda de política: agregué $\tau = 0$ y $\tau = +\infty$ con indicador de modo, el costo $(12FN + FP)/N$, la restricción de alertas, los desempates del 7d y el reporte de inviabilidad.
- `metricas_manuales` y `comparar_politicas` → celda de evaluación final: TP/TN/FP/FN con NumPy verificados con `confusion_matrix`; quité el recorte de probabilidades en la log-loss y agregué especificidad, Brier, costo y alertas por cada 1000 vuelos.

Son míos, sin código base del curso: `OLSRegression`, `RidgeRegression`, `numerical_gradient`, `relative_gradient_error`, las 30 pruebas del Notebook 1, la simulación del Problema 5 y los bucles de validación cruzada de los Problemas 6 y 7.

Contribuciones. Trabajo individual (Juan José Jiménez Maya): derivaciones, implementación, experimentos e informe.

Uso de IA generativa. Utilicé IA generativa como apoyo para encaminar las demostraciones (redacté el planteamiento original en Obsidian y utilicé Claude exclusivamente para transcribir y formatear el texto de

Markdown a LaTeX), y también la utilicé para adaptar lo necesario del código base de los talleres a las necesidades y requerimientos específicos de esta implementación. Ninguna salida de la IA sustituye las demostraciones, las pruebas numéricas o los resultados ejecutados, que se reproducen al ejecutar los notebooks desde un kernel limpio.

Referencias

[1] Curso de Aprendizaje Automático, UNAL Medellín (2026-2S). *Regresión lineal y regresión logística*, clase teórica de la semana 5 (diapositivas).

[2] Curso de Aprendizaje Automático (2026-2S). Notebook `Regresion_lineal_logistica_regularizacion.ipynb`.

[3] Curso de Aprendizaje Automático (2026-2S). Notebook `Clase_practica_EDA_para_Machine_Learning.ipynb`.

[4] Curso de Aprendizaje Automático (2026-2S). Notebook `Taller_clasificacion_vuelos.ipynb`.

[5] Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press (versión en línea de 2025), caps. 1–4.

[6] Brooks, T., Pope, D., Marcolini, M. (1989). *Airfoil Self-Noise*. UCI Machine Learning Repository, doi:10.24432/C5VW2C.

[7] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. *JMLR* 12, 2825–2830.